Лабораторная работа №1

Julia. Установка и настройка. Основные принципы

Ко Антон Геннадьевич

2026-09-26

Содержание I

1 Информация

Раздел 1

Информация

Докладчик

- Ко Антон Геннадьевич
- студент
- Российский университет дружбы народов им. П. Лумумбы
- 1132221551@rudn.ru
- https://SenDerMen04.github.io/ru/



1. Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

2. Выполнение лабораторной работы

[3]:
```
# Пустой кортеж:
()
```

[3]:
```
()
```

[4]:
```
# кортеж из элементов типа String:
favouritelang = ("Python", "Julia", "R")
```

[4]:
```
("Python", "Julia", "R")
```

[5]:
```
# кортеж из целых чисел:
x1 = (1, 2, 3)
```

[5]:
```
(1, 2, 3)
```

[6]:
```
# котреж из элементов разных типов:
x2 = (1, 2.0, "tmp")
```

[6]:
```
(1, 2.0, "tmp")
```

[7]:
```
# именованный котреж:
```

Примеры операций над кортежами (рис. #fig:002):

```
[8]: # длина котрежа x2:
     length(x2)

[8]: 3

[9]: # обратиться к элементам котрежа x2:
     x2[1], x2[2], x2[3]

[9]: (1, 2.0, "tmp")

[10]: # произвести какую-либо операцию (сложение)
      # с вторым и третьим элементами кортежа x1:
      c = x1[2] + x1[3]

[10]: 5

[11]: # обращение к элементам именованного кортежа x3:
      x3.a, x3.b, x3[2]

[11]: (2, 3, 3)

[12]: # проверка вхождения элементов tmp и 0 в кортеж x2
      # (два способа обращения к методу in()):
      in("tmp", x2), 0 in x2

[12]: (true, false)
```

Рисунок 2: Примеры операций над кортежами

Примеры словарей и операций над ними (рис. #fig:003):

```
  "Бухгалтерия"
  "Иванов И.И."
```

```
[16]: # вывести значения элементов словаря:
values(phonebook)
```

```
[16]: ValueIterator for a Dict{String, Any} with 2 entries. Values:
  "555-2368"
  ("867-5309", "333-5544")
```

```
[17]: # вывести заданные в словаре пары "ключ-значение":
pairs(phonebook)
```

```
[17]: Dict{String, Any} with 2 entries:
  "Бухгалтерия" => "555-2368"
  "Иванов И.И." => ("867-5309", "333-5544")
```

```
[18]: # проверка вхождения ключа в словарь:
haskey(phonebook, "Иванов И.И.")
```

```
[18]: true
```

```
[19]: # добавить элемент в словарь:
phonebook["Сидоров П.С."] = "555-3344"
```

```
[19]: "555-3344"
```

```
[20]: # удалить ключ и связанные с ним значения из словаря:
pop!(phonebook, "Иванов И.И.")
```

```
[20]: ("867-5309", "333-5544")
```

```
[21]: # Объединение словарей (функция merge()):
a = Dict("foo" => 0.0, "bar" => 42.0);
b = Dict("baz" => 17, "bar" => 13.0);
merge(a, b), merge(b, a)
```

Примеры множеств и операций над ними (рис. #fig:004 – #fig:005):

[22]:
```
# Множества

# Создать множество из четырёх целочисленных значений:
A = Set([1, 3, 4, 5])
```

[22]:
```
Set{Int64} with 4 elements:
  5
  4
  3
  1
```

[23]:
```
B = Set("Abrakadabra")
```

[23]:
```
Set{Char} with 6 elements:
  'a'
  'd'
  'A'
  'r'
  'k'
  'b'
```

[27]:
```
S1 = Set([1,2]);
S2 = Set([3,4]);
issetequal(S1,S2)
```

[27]:
```
false
```

[29]:
```
S3 = Set([1,2,2,3,1,2,3,2,1]);
S4 = Set([2,3,1]);
issetequal(S3,S4)
```

[29]:
```
true
```

[30]:
```
# Объединение множеств
C=union(S1,S2)
```

[30]:
```
Set{Int64} with 4 elements:
  4
  2
  3
  1
```

[31]:
```
# Пересечение множеств:
D = intersect(S1,S3)
```

[31]:
```
Set{Int64} with 2 elements:
  2
  1
```

[32]:
```
E = setdiff(S3,S1)
```

[32]:
```
Set{Int64} with 1 element:
  3
```

[33]:
```
issubset(S1,S4)
```

[33]:
```
true
```

[34]:
```
push!(S4, 99)
```

[34]:
```
Set{Int64} with 4 elements:
  2
  99
  3
  1
```

[35]:
```
pop!(S4)
```

[35]:
```
2
```

[36]:
```
# Массивы

empty_array_1 = []
```

[36]:
```
Any[]
```

[37]:
```
empty_array_2 = (Int64)[]
empty_array_3 = (Float64)[]
```

[37]:
```
Float64[]
```

[38]:
```
a = [1, 2, 3]
```

[38]:
```
3-element Vector{Int64}:
 1
 2
 3
```

[39]:
```
b = [1 2 3]
```

[39]:
```
1×3 Matrix{Int64}:
 1  2  3
```

[41]:
```
A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]
B = [[1 2 3]; [4 5 6]; [7 8 9]]
```

[41]:
```
3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8  9
```

[42]:
```
c = rand(1,8)
```

[42]:
```
1×8 Matrix{Float64}:
 0.852999  0.639707  0.405209  0.525749  …  0.0333724  0.573043  0.781425
```

[45]:
```
C = rand(2,3)
```

[45]:
```
2×3 Matrix{Float64}:
 0.887847  0.959955  0.397307
 0.531252  0.551835  0.781073
```

[46]:
```
D = rand(4, 3, 2)
```

[46]:
```
4×3×2 Array{Float64, 3}:
[:, :, 1] =
 0.564331  0.182363   0.498387
 0.660128  0.961337   0.446801
 0.741215  0.0215336  0.0604697
 0.396765  0.458156   0.405614

[:, :, 2] =
 0.12664     0.131427  0.56012
 0.195112    0.122124  0.974208
 0.321852    0.529345  0.428506
 0.00257362  0.762855  0.493766
```

Примеры массивов, заданных некоторыми функциями через включение (рис. #fig:008):

```
[47]: roots = [sqrt(i) for i in 1:10]
```

```
[47]: 10-element Vector{Float64}:
 1.0
 1.4142135623730951
 1.7320508075688772
 2.0
 2.23606797749979
 2.449489742783178
 2.6457513110645907
 2.8284271247461903
 3.0
 3.1622776601683795
```

```
[48]: ar_1 = [3*i^2 for i in 1:2:9]
```

```
[48]: 5-element Vector{Int64}:
   3
  27
  75
 147
 243
```

```
[49]: ar_2 = [i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]
```

```
[49]: 4-element Vector{Int64}:
  1
  9
 49
```

Некоторые операции для работы с массивами (рис. #fig:009 – #fig:014):

```
[50]: ones(5)
```

```
[50]: 5-element Vector{Float64}:
       1.0
       1.0
       1.0
       1.0
       1.0
```

```
[51]: ones(2,3)
```

```
[51]: 2×3 Matrix{Float64}:
       1.0  1.0  1.0
       1.0  1.0  1.0
```

```
[52]: zeros(4)
```

```
[52]: 4-element Vector{Float64}:
       0.0
       0.0
       0.0
       0.0
```

```
[54]: fill(3.5,(3,2))
```

```
[54]: 3×2 Matrix{Float64}:
       3.5  3.5
```

[55]:
```
repeat([1,2],3,3)
repeat([1 2],3,3)
```

[55]:
```
3×6 Matrix{Int64}:
 1  2  1  2  1  2
 1  2  1  2  1  2
 1  2  1  2  1  2
```

[56]:
```
a = collect(1:12)
b = reshape(a,(2,6))
```

[56]:
```
2×6 Matrix{Int64}:
 1  3  5  7   9  11
 2  4  6  8  10  12
```

[57]:
```
b'
```

[57]:
```
6×2 adjoint(::Matrix{Int64}) with eltype Int64:
  1   2
  3   4
  5   6
  7   8
  9  10
 11  12
```

[58]:
```
c = transpose(b)
```

[58]:
```
6×2 transpose(::Matrix{Int64}) with eltype Int64:
  1   2
  3   4
  5   6
  7   8
  9  10
 11  12
```

[59]:
```
ar = rand(10:20, 10, 5)
```

[59]:
```
10×5 Matrix{Int64}:
 13  19  20  20  18
 12  20  15  13  13
 10  10  20  16  18
 14  20  17  11  13
 16  11  11  14  12
 20  14  17  15  10
 15  14  19  12  17
 17  15  15  19  13
 13  11  14  19  11
 12  16  14  11  17
```

[60]:
```
ar[:, 2]
```

[60]:
```
10-element Vector{Int64}:
 19
 20
 10
 20
 11
 14
 14
 15
 11
 16
```

[61]:
```
ar[:, [2, 5]]
```

[61]:
```
10×2 Matrix{Int64}:
 19  18
 20  13
 10  18
 20  13
 11  12
 14  10
 14  17
 15  13
 11  11
 16  17
```

[62]:
```
ar[:, 2:4]
```

[62]:
```
10×3 Matrix{Int64}:
 19  20  20
 20  15  13
 10  20  16
 20  17  11
 11  11  14
 14  17  15
 14  19  12
 15  15  19
 11  14  19
 16  14  11
```

[63]:
```
ar[[2, 4, 6], [1,5]]
```

[63]:
```
3×2 Matrix{Int64}:
 12  13
 14  13
 20  10
```

```
[64]: ar[1, 3:end]
```

```
[64]: 3-element Vector{Int64}:
 20
 20
 18
```

```
[65]: sort(ar,dims=1)
```

```
[65]: 10×5 Matrix{Int64}:
 10  10  11  11  10
 12  11  14  11  11
 12  11  14  12  12
 13  14  15  13  13
 13  14  15  14  13
 14  15  17  15  13
 15  16  17  16  17
 16  19  19  19  17
 17  20  20  19  18
 20  20  20  20  18
```

```
[66]: sort(ar,dims=2)
```

```
[66]: 10×5 Matrix{Int64}:
 13  18  19  20  20
 12  13  13  15  20
 10  10  16  18  20
 11  13  14  17  20
 11  11  12  14  16
 10  14  15  17  20
 12  14  15  17  19
 13  15  15  17  19
 11  11  13  14  19
 11  12  14  16  17
```

```
[67]: ar .> 14
```

```
[67]: 10×5 BitMatrix:
 0  1  1  1  1
 0  1  1  0  0
 0  0  1  1  1
 0  1  1  0  0
 1  0  0  0  0
 1  0  1  1  0
 1  0  1  0  1
 1  1  1  1  0
 0  0  0  1  0
 0  1  0  0  1
```

```
[68]: findall(ar .> 14)
```

```
[68]: 25-element Vector{CartesianIndex{2}}:
 CartesianIndex(5, 1)
 CartesianIndex(6, 1)
 CartesianIndex(7, 1)
 CartesianIndex(8, 1)
 CartesianIndex(1, 2)
 CartesianIndex(2, 2)
 CartesianIndex(4, 2)
 CartesianIndex(8, 2)
 CartesianIndex(10, 2)
 CartesianIndex(1, 3)
 CartesianIndex(2, 3)
 CartesianIndex(3, 3)
 CartesianIndex(4, 3)
 CartesianIndex(6, 3)
 CartesianIndex(7, 3)
 CartesianIndex(8, 3)
 CartesianIndex(1, 4)
 CartesianIndex(3, 4)
 CartesianIndex(6, 4)
 CartesianIndex(8, 4)
 CartesianIndex(9, 4)
 CartesianIndex(1, 5)
 CartesianIndex(3, 5)
```

2.5 Самостоятельная работа

Выполнение заданий №1 и №2 (рис. #fig:015):

[69]:
```
# Задание 1.

A = Set([0, 3, 4, 9])
B = Set([1, 3, 4, 7])
C = Set([0, 1, 2, 4, 7, 8, 9])

P = union(intersect(A, B), intersect(A, C), intersect(B, C))
println(P)
```

```
Set([0, 4, 7, 9, 3, 1])
```

[71]:
```
# Задание 2

set1 = Set(["Toyota", "Mazda", "Honda"])
set2 = Set(["Mazda", "Honda", "Ford"])
intersection = intersect(set1, set2)
println(intersection)
```

```
Set(["Mazda", "Honda"])
```

[72]:
```
set3 = Set([50, 80, 90])
set4 = Set([90, 20, 40])
difference = setdiff(set3, set4)
println(difference)
```

Выполнение задания №3 (всех подпунктов) (рис. #fig:016 – #fig:020):

[73]:
```
# Задание 3.1

N = 28
array1 = collect(1:N)
println(array1)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28]
```

[74]:
```
# Задание 3.2

N = 28
array1 = collect(N:-1:1)
println(array1)
```

```
[28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

[75]:
```
# Задание 3.3

N = 28
array2 = vcat(collect(1:N), collect(N-1:-1:1))
println(array2)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16,
15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

[76]:
```
tmp = [4, 6, 3]
println(tmp)
```

```
[4, 6, 3]
```

Рисунок 16: Выполнение подпунктов задания №3

[4, 6, 3]

[84]:
```
# Задание 3.5
array4 = fill(tmp[1], 10)
println(array4)
```

```
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
```

[85]:
```
# Задание 3.6
array5 = repeat(tmp, 10)
println(array5)
```

```
[4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3]
```

[86]:
```
# Задание 3.7
array6 = vcat(fill(tmp[1], 11), fill(tmp[2], 10), fill(tmp[3], 10))
println(array6)
```

```
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 3, 3, 3, 3, 3, 3, 3, 3, 3]
```

[87]:
```
# Задание 3.8
array7 = vcat(fill(tmp[1], 10), fill(tmp[2], 20), fill(tmp[3], 30))
println(array7)
```

```
[4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 3, 3, 3, 3, 3, 3, 3, 3, 3
```

[89]:
```
# Задание 3.9

tmp = [4, 6, 3]

res_array = [2^tmp[i] for i in 1:3]
res_array = vcat(res_array, repeat([2^tmp[3], 4]))

res_string = join(res_array, "")
count_6 = count(x -> x == '6', res_string)

println("Массив: ", res_array)
println("Кол-во 6: ", count_6)
```

```
Массив: [16, 64, 8, 8, 4]
Кол-во 6: 2
```

[91]:
```
# Задание 3.10

using Statistics

x = 3:0.1:6
y = [exp(x) * cos(x) for x in x]

println("Сред. y ", mean(y))
```

```
Сред. y 53.11374594642971
```

Рисунок 18: Выполнение подпунктов задания №3

[92]:
```
# Задание 3.11

xi = collect(3:3:36) * 0.1
yj = collect(1:3:34) * 0.2
vector1 = [(x, y) for x in xi, y in yj]
println(vector1)
```

```
[(0.30000000000000004, 0.2) (0.30000000000000004, 0.8) (0.30000000000000004, 1.4000000000000001) (0.30000000000000004, 2.0) (0.30000000000000004, 2.6) (0.30000000000000004, 3.2) (0.30000000000000004, 3.8000000000000003) (0.30000000000000004, 4.4) (0.30000000000000004, 5.0) (0.30000000000000004, 5.6000000000000005) (0.30000000000000004, 6.2) (0.30000000000000004, 6.800000000000001); (0.6000000000000001, 0.2) (0.6000000000000001, 0.8) (0.6000000000000001, 1.4000000000000001) (0.6000000000000001, 2.0) (0.6000000000000001, 2.6) (0.6000000000000001, 3.2) (0.6000000000000001, 3.8000000000000003) (0.6000000000000001, 4.4) (0.6000000000000001, 5.0) (0.6000000000000001, 5.6000000000000005) (0.6000000000000001, 6.2) (0.6000000000000001, 6.800000000000001); (0.9, 0.2) (0.9, 0.8) (0.9, 1.4000000000000001) (0.9, 2.0) (0.9, 2.6) (0.9, 3.2) (0.9, 3.8000000000000003) (0.9, 4.4) (0.9, 5.0) (0.9, 5.6000000000000005) (0.9, 6.2) (0.9, 6.800000000000001); (1.2000000000000002, 0.2) (1.2000000000000002, 0.8) (1.2000000000000002, 1.4000000000000001) (1.2000000000000002, 2.0) (1.2000000000000002, 2.6) (1.2000000000000002, 3.2) (1.2000000000000002, 3.8000000000000003) (1.2000000000000002, 4.4) (1.2000000000000002, 5.0) (1.2000000000000002, 5.6000000000000005) (1.2000000000000002, 6.2) (1.2000000000000002, 6.800000000000001); (1.5, 0.2) (1.5, 0.8) (1.5, 1.4000000000000001) (1.5, 2.0) (1.5, 2.6) (1.5, 3.2) (1.5, 3.8000000000000003) (1.5, 4.4) (1.5, 5.0) (1.5, 5.6000000000000005) (1.5, 6.2) (1.5, 6.800000000000001); (1.8, 0.2) (1.8, 0.8) (1.8, 1.4000000000000001) (1.8, 2.0) (1.8, 2.6) (1.8, 3.2) (1.8, 3.8000000000000003) (1.8, 4.4) (1.8, 5.0) (1.8, 5.6000000000000005) (1.8, 6.2) (1.8, 6.800000000000001); (2.1, 0.2) (2.1, 0.8) (2.1, 1.4000000000000001) (2.1, 2.0) (2.1, 2.6) (2.1, 3.2) (2.1, 3.8000000000000003) (2.1, 4.4) (2.1, 5.0) (2.1, 5.6000000000000005) (2.1, 6.2) (2.1, 6.800000000000001); (2.4000000000000004, 0.2) (2.4000000000000004, 0.8) (2.4000000000000004, 1.4000000000000001) (2.4000000000000004, 2.0) (2.4000000000000004, 2.6) (2.4000000000000004, 3.2) (2.4000000000000004, 3.8000000000000003) (2.4000000000000004, 4.4) (2.4000000000000004, 5.0) (2.4000000000000004, 5.6000000000000005) (2.4000000000000004, 6.2) (2.4000000000000004, 6.800000000000001); (2.7, 0.2) (2.7, 0.8) (2.7, 1.4000000000000001) (2.7, 2.0) (2.7, 2.6) (2.7, 3.2) (2.7, 3.8000000000000003) (2.7, 4.4) (2.7, 5.0) (2.7, 5.6000000000000005) (2.7, 6.2) (2.7, 6.800000000000001); (3.0, 0.2) (3.0, 0.8) (3.0, 1.4000000000000001) (3.0, 2.0) (3.0, 2.6) (3.0, 3.2) (3.0, 3.8000000000000003) (3.0, 4.4) (3.0, 5.0) (3.0, 5.6000000000000005) (3.0, 6.2) (3.0, 6.800000000000001); (3.3000000000000003, 0.2) (3.3000000000000003, 0.8) (3.3000000000000003, 1.4000000000000001) (3.3000000000000003, 2.0) (3.3000000000000003, 2.6) (3.3000000000000003, 3.2) (3.3000000000000003, 3.8000000000000003) (3.3000000000000003, 4.4) (3.3000000000000003, 5.0) (3.3000000000000003, 5.6000000000000005) (3.3000000000000003, 6.2) (3.3000000000000003, 6.800000000000001); (3.6, 0.2) (3.6, 0.8) (3.6, 1.4000000000000001) (3.6, 2.0) (3.6, 2.6) (3.6, 3.2) (3.6, 3.8000000000000003) (3.6, 4.4) (3.6, 5.0) (3.6, 5.6000000000000005) (3.6, 6.2) (3.6, 6.800000000000001)]
```

[93]:
```
# Задание 3.12

M = 25
vector2 = [2^i / i for i in 1:M]
println(vector2)
```

```
[2.0, 2.0, 2.6666666666666665, 4.0, 6.4, 10.666666666666666, 18.285714285714285, 32.0, 56.888888888888886, 102.4, 186.18181818181818, 341.3333333333333
```

[94]:
```julia
# Задание 3.13

N = 30
vector3 = ["fn$i" for i in 1:N]
println(vector3)
```

```
["fn1", "fn2", "fn3", "fn4", "fn5", "fn6", "fn7", "fn8", "fn9", "fn10", "fn11", "fn12", "fn13", "fn14", "fn15", "fn16", "fn17", "fn18", "fn19", "fn20",
"fn21", "fn22", "fn23", "fn24", "fn25", "fn26", "fn27", "fn28", "fn29", "fn30"]
```

[95]:
```julia
# Задание 3.14

using Random

x = rand(0:999, 250)
y = rand(0:999, 250)

filtered_y = y[y .> 600]
println("Элементы y > 600: ", filtered_y)

corresponding_x = x[findall(y .> 600)]
println("Соответ. x: ", corresponding_x)

sum_exp = sum(exp.(-x[2:end] + x[1:end-1]) .+ 10)
println("Сумма: ", sum_exp)

unique_x = unique(x)
println("Уник. элем. x: ", unique_x)
```

```
Элементы y > 600: [941, 710, 923, 733, 865, 858, 759, 908, 761, 732, 952, 774, 812, 980, 861, 786, 907, 985, 645, 796, 699, 868, 665, 674, 709, 913, 84
7, 720, 823, 712, 691, 611, 975, 915, 730, 826, 615, 836, 959, 848, 677, 749, 713, 744, 657, 664, 907, 603, 991, 876, 705, 880, 812, 847, 700, 894, 63
4, 799, 908, 947, 659, 853, 748, 927, 618, 728, 725, 676, 705, 788, 621, 683, 955, 933, 796, 720, 700, 862, 852, 688, 844, 650, 834, 762, 778, 940, 78
0, 717, 724, 631, 738, 833, 689, 869, 676, 756, 900, 874, 712, 841]
Соответ. x: [25, 49, 699, 879, 530, 942, 500, 292, 972, 682, 336, 604, 213, 167, 578, 380, 2, 476, 650, 232, 257, 376, 293, 515, 897, 626, 899, 38, 32,
672, 269, 493, 317, 391, 378, 84, 906, 233, 531, 684, 54, 696, 855, 385, 30, 290, 779, 337, 360, 450, 392, 643, 749, 618, 511, 511, 321, 525, 590, 450,
488, 660, 854, 14, 304, 296, 473, 745, 659, 845, 804, 893, 913, 605, 241, 373, 10, 439, 760, 949, 234, 237, 786, 52, 160, 144, 422, 396, 321, 910, 649,
333, 731, 754, 244, 186, 737, 107, 767, 973]
Сумма: Inf
Уник. элем. x: [25, 960, 694, 350, 187, 49, 900, 974, 699, 879, 206, 530, 942, 500, 26, 292, 509, 972, 992, 477, 30, 397, 682, 495, 950, 336, 604, 213,
```

Решение заданий №4 и №6 (рис. #fig:021):

[96]:
```
# Задание 4

squares = [i^2 for i in 1:100]
println(squares)
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400, 441, 484, 529, 576, 625, 676, 729, 784, 841, 900, 961, 1024, 1089, 1156, 1225, 1296, 1369, 1444, 1521, 1600, 1681, 1764, 1849, 1936, 2025, 2116, 2209, 2304, 2401, 2500, 2601, 2704, 2809, 2916, 3025, 3136, 3249, 3364, 3481, 3600, 3721, 3844, 3969, 4096, 4225, 4356, 4489, 4624, 4761, 4900, 5041, 5184, 5329, 5476, 5625, 5776, 5929, 6084, 6241, 6400, 6561, 6724, 6889, 7056, 7225, 7396, 7569, 7744, 7921, 8100, 8281, 8464, 8649, 8836, 9025, 9216, 9409, 9604, 9801, 10000]
```

[98]:
```
# Задание 6

sum1 = sum(i^3 + 4*i^2 for i in 10:100)
println(sum1)

sum2 = sum((2^i / i + 3^i / i^2) for i in 1:25)
println(sum2)

sum3 = sum(prod(2:2n) / prod(3:2n+1) for n in 19:19)
println(sum3)
```

```
26852735
2.1291704368143802e9
4.156989266329373
```

Рисунок 21: Решение заданий №4 и №6

3. Вывод

В ходе выполнения лабораторной работы были изучены несколько структур данных, реализованных в Julia, а также научились применять их и операции над ними для решения задач.